

■ Terminologia

::, operador unário de resolução de escopo 457
 análise de um documento de requisitos 464
 análise e projeto orientados a objeto (OOAD) 465
 argumento default 455
 assinaturas de uma função 459
 associações (na UML) 464
 atributos de um objeto 463
 biblioteca-padrão de C++ 445
 bool, palavra-chave 449
 cabeçalho de stream de entrada/saída (<iostream>) 444
 classes 445
 clientes de uma classe 464
 comportamentos de um objeto 463
 dado-membro de uma classe 464
 decoração de nome 459
 deformação de nome 459
 encapsulamento 463
 endl, manipulador de stream 445
 especialização de templates de função 461
 false, palavra-chave 449
 função inline 448
 funções-membro 464
 herança 463
 inline, palavra-chave 448
 interfaces 463
 ligação segura quanto a tipos 451
 linguagem de programação procedural 464
 linguagem extensível 465
 lista de parâmetros de template 461
 manipulador de stream 445
 métodos de uma classe 464
 Object Management Group (OMG) 465
 objetos 463
 objeto de stream de entrada-padrão (cin) 445
 objeto de stream de saída-padrão (cout) 445
 ocultação de informações 463
 operações de inserção de stream por cascata 445
 operações de inserção de stream por concatenação 445
 operações de inserção de stream por encadeamento 445
 operações de uma classe 463
 operador de extração de stream (>>) 445
 operador de inserção de stream (<<) 445
 orientação a ação 464
 parâmetros de referência 450
 parâmetros de tipo formal 461
 programação genérica 443
 programação orientada a objeto (OOP) 443
 projeto orientado a objeto (OOD) 463
 referências penduradas 454
 requisitos de um projeto 464
 reutilização de software 446
 sobrecarga de função 458
 sobrecarga de operador 445
 templates de função 461
 true, palavra-chave 449
 UML Partners 465
 Unified Modeling Language (UML) 463

■ Exercícios de autorrevisão

15.1 Preencha os espaços em cada uma das sentenças:

- a)** Em C++, é possível ter várias funções com o mesmo nome, que operam sobre diferentes tipos ou números de argumentos. Isso é chamado _____ de função.
- b)** O(A) _____ habilita o acesso a uma variável global com o mesmo nome de uma variável no escopo atual.
- c)** Um(a) _____ de função permite que uma única função seja definida para realizar a mesma tarefa sobre dados de muitos tipos diferentes.
- d)** _____ é o esquema de representação gráfica mais usado para a modelagem OO.
- e)** _____ modela componentes de software em termos de objetos do mundo real.

f) Programadores C++ criam seus próprios tipos definidos pelo usuário, chamados _____.

15.2 Por que um protótipo de função contém uma declaração de tipo de parâmetro como `double &`?

15.3 (Verdadeiro/Falso) Todos os argumentos de chamadas de função em C++ são passados por valor.

15.4 Escreva um programa completo que peça ao usuário que forneça o raio de uma esfera e calcule e imprima o volume dessa esfera. Use uma função `inline sphereVolume` que retorne o resultado da seguinte expressão:

`( 4.0 / 3.0 ) * 3.14159 * pow( radius, 3 ).`

■ Respostas dos exercícios de autorrevisão

- 15.1** a) sobrecarga. b) operador unário de resolução de escopo (`::`). c) template. d) A UML. e) Projeto orientado a objeto (OOD). f) classes.
- 15.2** Isso cria um parâmetro de referência do tipo “referência para `double`”, que permite que a função modifique a variável original na função que chamou.
- 15.3** Falso. C++ permite passagem por referência usando parâmetros de referência.
- 15.4** Veja o programa a seguir:

```

1 // Exercício 15.4 Solução: Ex15_04.cpp
2 // Função inline que calcula o volume de
  uma esfera.
3 #include <iostream>
4 #include <cmath>
5
6 const double PI = 3.14159; // define
  constante global PI
7
8 // calcula volume de uma esfera
9 inline double sphereVolume( const double
  radius )

```

```

10 {
11     return 4.0 / 3.0 * PI * pow( radius, 3
12 );
13 } // fim da função inline sphereVolume
14
15 int main()
16 {
17     double radiusValue;
18
19     // pede que o usuário informe o raio
20     cout << "Digite o tamanho do raio da
21     sua esfera: ";
22     cin >> radiusValue; // insere radius
23
24     // usa radiusValue para calcular volu-
25     me da esfera e exibe resultado
26     cout << "Volume da esfera com raio "
27     << radiusValue
28     << " é " << sphereVolume( radiusVa-
29     lue ) << endl;
30 } // fim do main

```

Digite o tamanho do raio da sua esfera: 2
Volume da esfera com raio 2 é 33.5103

■ Exercícios

- 15.5** Escreva um programa em C++ que peça ao usuário que forneça o raio de um círculo, e depois chame a função `inline circleArea` para calcular a área desse círculo.
- 15.6** Escreva um programa completo em C++ com duas funções alternativas especificadas a seguir, cada uma simplesmente triplicando a variável `count` definida em `main`. Depois, compare as duas técnicas. Essas duas funções são:
- a) a função `tripleByValue`, que passa uma cópia de `count` por valor, triplica a cópia e retorna o novo valor, e
 - b) a função `tripleByReference`, que passa `count` por referência por meio de um parâmetro de referência e triplica o valor original de `count` por meio de seu alias (ou seja, o parâmetro de referência).
- 15.7** Qual é a finalidade do operador unário de resolução de escopo?
- 15.8** Escreva um programa que use um template de função chamado `min` para determinar o menor de dois argumentos. Teste o programa usando argumentos com números inteiros, de caracteres e de ponto flutuante.
- 15.9** Escreva um programa que use um template de função chamado `max` para determinar o maior de dois argumentos. Teste o programa usando argumentos com números inteiros, de caracteres e de ponto flutuante.
- 15.10** Determine se os segmentos de programa a seguir contêm erros. Explique como cada erro pode ser corrigido. [Nota: para um segmento de programa em particular, é possível que nenhum erro esteja presente no segmento.]
- a)

```
template < class A >
int sum( int num1, int num2, int num3 )
{
    return num1 + num2 + num3;
}
```
 - b)

```
void printResults( int x, int y )
{
    cout << "A soma é " << x + y << '\n';
    return x + y;
}
```
 - c)

```
template < A >
A product( A num1, A num2, A num3 )
{
    return num1 * num2 * num3;
}
```
 - d)

```
double cube( int );
int cube( int );
```